



8. P00: Herència

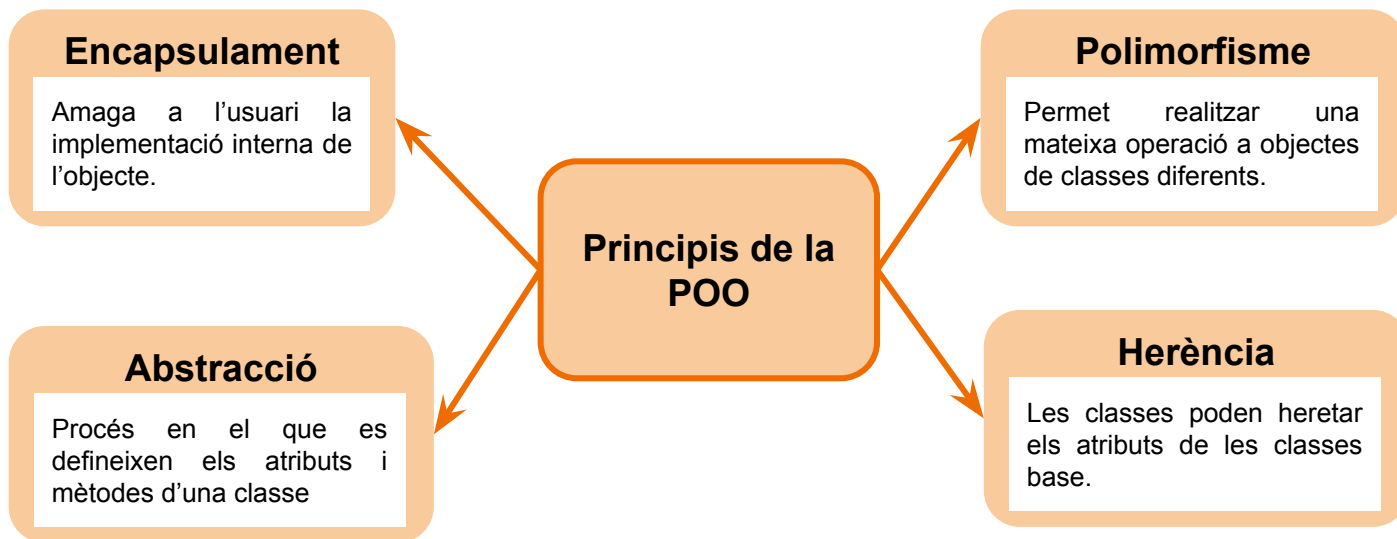
RA2, RA4, RA7

MP0485 - Programació

Curs 2025-2026



1. Recordem els 4 principis de la POO...





2. Recordem: classes i objectes

Una **classe** guarda una abstracció del que representa un objecte. És com una *plantilla* que **defineix com seran els objectes** d'un tipus concret.

Amb una classe podem instanciar tants objectes com vulguem.

Què caracteritza un cotxe?

- marca
- model
- color
- nombre de portes
- nombre de places
- descapotable
- ...





3. Herència

L'**herència** ens permet reutilitzar codi, creant classes que deriven d'altres.

Imaginem que volem desenvolupar un videojoc on hi ha vehicles:

- cotxes
- motos
- bicicletes

Aquests objectes **comparteixen forces característiques**, però altres són pròpies.



4. Classes Cotxe i Motocicleta





4. Classes Cotxe i Motocicleta



- preu
- model
- color
- nombre de portes
- nombre de places
- descapotable

- encendre motor
- apagar motor
- accelerar
- frenar
- ...



- preu
- model
- color
- nombre de portes
- nombre de places

- encendre motor
- apagar motor
- accelerar
- frenar
- fer un cavallet
- ...



4. Classes Cotxe i Motocicleta



Cotxe
- propietari - color - numportes - numplaces - motor - velocitat - descapotable
+ encendreMotor() + apagarMotor() + accelerar() + frenar()



Moto
- propietari - color - numportes - numplaces - motor - velocitat
+ encendreMotor() + apagarMotor() + accelerar() + frenar() + ferCavallet()

4. Classes Cotxe i Motocicleta



Cotxe
- propietari - color - num_portes - num_places - motor - velocitat - descapotable
+ encendreMotor() + apagarMotor() + accelerar() + frenar()



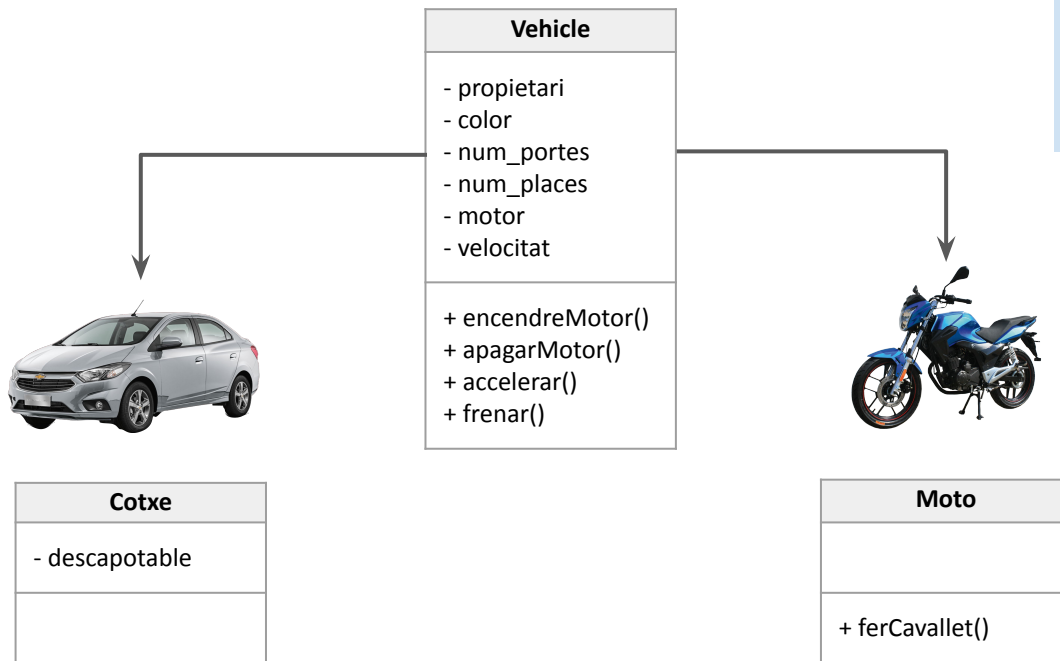
Moto
- propietari - color - num_portes - num_places - motor - velocitat
+ encendreMotor() + apagarMotor() + accelerar() + frenar() + ferCavallet()

Comparteixen molts mètodes i atributs, pel que podríem decidir fer **una classe comuna**.





5. Classe base: Vehicle



La **classe base** o principal defineix tots els atributs i mètodes que tenen en comú la resta de subclasses o **classes filles**.



6. Classe base

A la **classe base** o principal definirem tots els atributs i mètodes que tenen en comú la resta de subclasses o **classes filles**.

La **classe base**:

- **Estructura:** s'estructura i crea igual que les classes vistes anteriorment.
- **Atributs i mètodes:** Contindrà tots els atributs i mètodes que siguin comuns a les classes filles.
- **Sobreescriptura:** Els mètodes comuns es poden definir i redefinir a les classes filles.
- **Visibilitat:** És recomanable que els atributs tinguin **visibilitat protegida** (protected) per tal de que siguin visibles a les classes filles. Per fer-ho, s'indica amb un guionet baix davant del nom de l'atribut.

```
class ClasseBase:
    def __init__(self, atribut_base):
        self.atribut_base = atribut_base

    def metode_base(self):
        print("Mètode de la classe base")
```

```
class ClasseDerivada(ClasseBase):
    def __init__(self, atribut_base, atribut_propi):
        # Crida al constructor de la classe base
        super().__init__(atribut_base)
        self.atribut_propi = atribut_propi

    def metode_propi(self):
        print("Mètode propi de la classe")
```



7. Les classes filles

Cada **classe filla** o derivada serà una extensió de la classe base. Per tant, heretarà els atributs i mètodes d'aquesta, i a més podrà definir els seus propis.

La **classe filla**:

- **Herència:** Heretarà tots els atributs i mètodes de la classe base **sense haver de tornar a escriure el codi**. Només cal indicar d'on hereta:
class nomClasseFilla (nomClasseBase).
- **Mètode super():** Cal cridar al constructor de la classe base, fent servir el mètode super().
- **Sobreescriptura:** Els mètodes de la classe base es poden redefinir a les classes filles si es desitja (overriding).

```
class ClasseBase:
    def __init__(self, atribut_base):
        self.atribut_base = atribut_base

    def metode_base(self):
        print("Mètode de la classe base")
```

```
class ClasseDerivada(ClasseBase):
    def __init__(self, atribut_base, atribut_propi):
        # Crida al constructor de la classe base
        super().__init__(atribut_base)
        self.atribut_propi = atribut_propi

    def metode_base(self):
        print("Mètode modificat per la classe filla")

    def metode_propi(self):
        print("Mètode propi de la classe")
```

8. Tornem a l'exemple

Vehicle de motor

```
class Vehicle_motor:
    #Constructor
    def __init__(self,color, num_rodes, velocitat, motor, propietari):
        self.color = color #string
        self.num_rodes = num_rodes #int
        self.velocitat = velocitat #float
        self.motor = motor #boolean encès/apagat
        self.__propietari = propietari #string

    #getters i setters

    def encendreMotor(self):
        self.motor = True

    def apagarMotor(self):
        self.motor = False

    def accelerar(self, valor):
        self.velocitat = velocitat+valor

    def frenar(self, valor):
        self.velocitat = velocitat-valor

    def __str__(self):
        return f"Vehicle de color {self.color}"
```





8. Tornem a l'exemple

```
class Cotxe(Vehicle_motor):  
    # Constructor  
    def __init__(self,color, num_rodes, velocitat, motor, propietari, descapotable):  
        super().__init__(color, num_rodes, velocitat, motor, propietari)  
        self.descapotable = descapotable  
  
    def get_descapotable(self):  
        return descapotable  
  
    def set_descapotable(self, descapotable):  
        self.descapotable = descapotable  
  
    def __str__(self):  
        return f"Cotxe de color {self.color} i descapotable {self.descapotable}"
```



8. Classes abstractes

Les **classes abstractes** representen alguna cosa que no es pot identificar amb quelcom concret, però sí engloben unes certes característiques.

Instanciació:

- No es poden instanciar objectes d'una classe abstracta.
- Cal, per tant, definir classes filles per crear-ne.

Mètodes abstractes:

- Són mètodes que **no s'implementen** a la classe base.
- **Cal implementar-los** obligatòriament a les **classes derivades**.
- No tots els mètodes d'una classe abstracte han de ser-ho.
- Les classes no abstractes també en poden tenir.

Classe abstracta Figura

```
public abstract class Figura { //Definim la classe abstracta Figura

    private String color;
    private double x;
    private double y;

    public Figura(String color, double x, double y) {
        this.color = color;
        this.x = x;
        this.y = y;
    }

    public String getColor() {
        return color;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public abstract double area(); //Definim el mètode abstracte area

    public void imprimirPosició() { //Implementem el mètode imprimirPosició
        System.out.println("Posició: (" + x + ", " + y + ")");
    }
}
```



Classe filla Cercle

```
//Definim la classe Cercle que hereta de Figura
public class Cercle extends Figura {

    private double radi;

    public Cercle(String color, double x, double y, double radi) {
        super(color, x, y);
        this.radi = radi;
    }

    public double getRadi() {
        return radi;
    }

    //Sobreescrivim el mètode abstracte area per calcular l'àrea d'un cercle
    @Override
    public double area() {
        return Math.PI * radi * radi;
    }

    // No cal implementar el mètode imprimirPosicio(),
    //ja que ja està implementat en la classe Figura
}
```

Classe filla Rectangle

```
//Definim la classe Rectangle que hereta de Figura
public class Rectangle extends Figura {

    private double base;
    private double altura;

    public Rectangle(String color, double x, double y, double base, double altura) {
        super(color, x, y);
        this.base = base;
        this.altura = altura;
    }

    public double getBase() {
        return base;
    }

    public double getAltura() {
        return altura;
    }

    //Sobreescrivim el mètode abstracte area per calcular l'àrea d'un rectangle
    @Override
    public double area() {
        return base * altura;
    }

    //No cal implementar el mètode imprimirPosicio(),
    //ja que ja està implementat en la classe Figura
}
```



Webgrafia

<https://ellibrodepython.com/herencia-en-python>

□ **CC BY 4.0**

Contingut elaborat per Carles Bonet Papell i Cristina Reina León.

<https://creativecommons.org/licenses/by/4.0/>